

ALU KATA

- 카타 (일본어)

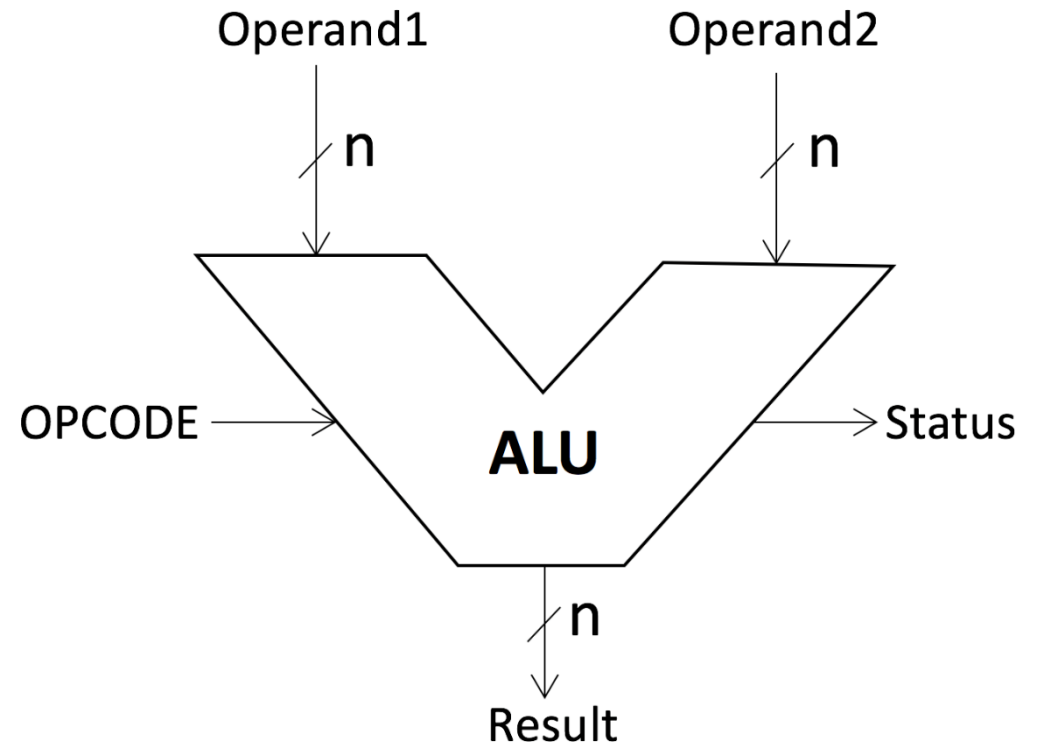
무술이 실전에서 바로 사용될 수 있도록,
상황을 가정하여 실무를 위한 시뮬레이션

- 리팩토링 카타

실무 리팩토링을 할 수 있도록
리팩토링 훈련 자료들

Operand1 과 Operand2에
수를 넣고,
OPCODE를 넣으면

그 결과로
Status / Result 가 나온다.



Lagacy Code

- 소스코드 링크

<https://github.com/mincoding1/ALU>



The screenshot shows a GitHub repository page for a file named `ALU.java` in the `main` branch. The file is 70 lines long, 65 lines of code, and 2.1 KB in size. The code is written in Java and defines a `public class ALU` with several methods: `setOperand1`, `setOperand2`, `setOPCODE`, and `enableSignal`. The `enableSignal` method contains a conditional logic block for performing an addition operation based on the `OPCODE` and operand values.

```
1  public class ALU {
2
3      private int operand1 = -1;
4      private int operand2 = -1;
5      private String OPCODE = "";
6
7      public void setOperand1(int operand1) {
8          this.operand1 = operand1;
9      }
10
11     public void setOperand2(int operand2) {
12         this.operand2 = operand2;
13     }
14
15     public void setOPCODE(String OPCODE) {
16         this.OPCODE = OPCODE;
17     }
18
19     public void enableSignal(Result r) {
20         if (OPCODE.equals("ADD") == true && !OPCODE.equals("MUL")) {
21             if (operand1 != -1 && operand2 != -1) {
22                 int result = operand1 + operand2;
23                 r.setResult(result);
24                 r.setStatus(0);
25             }
26             else if (operand1 == -1) {
```

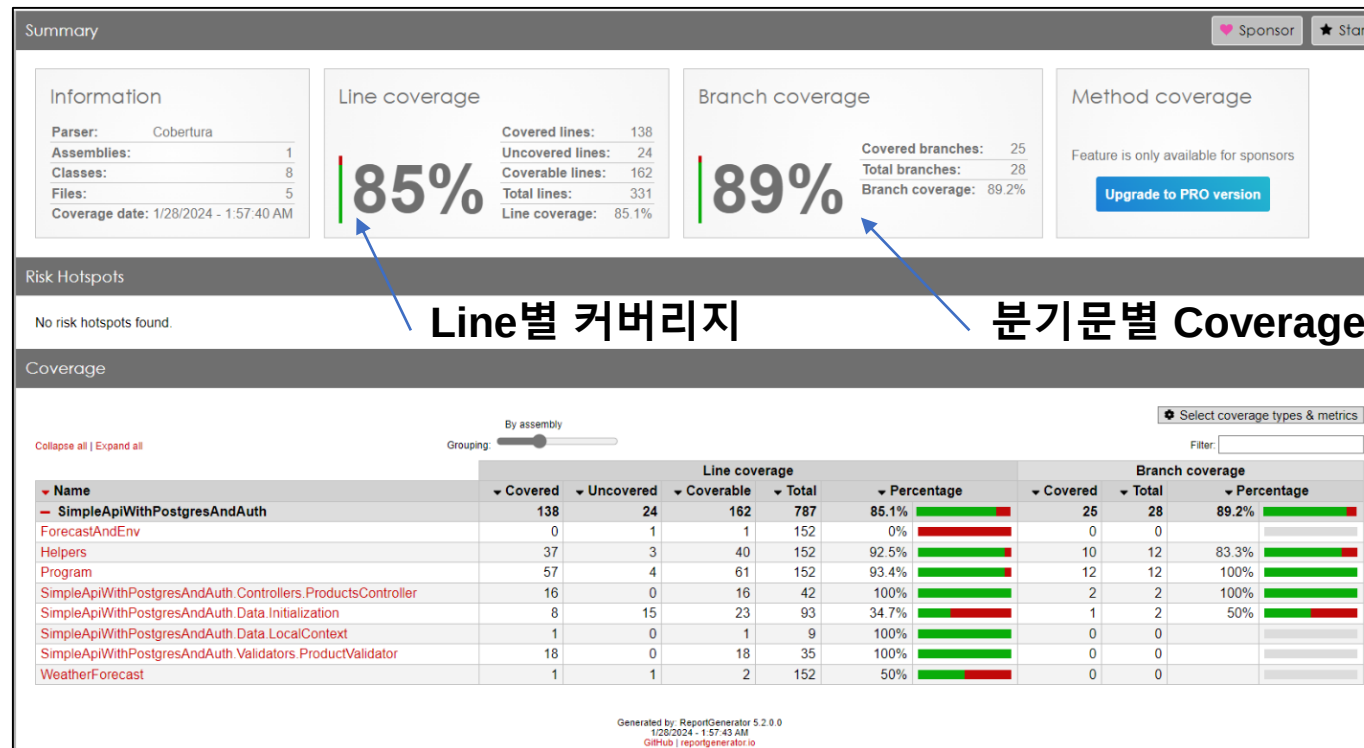
리팩토링 하기 전,

테스트 환경을 만들고 진행해야한다.
마음껏 리팩토링을 할 수 있다.

Code Coverage란?

테스트가 소스코드의 얼마나 **Touch** 했는지 측정할 수 있는 지표

- 테스트 코드가 어느 부분을 닿았는지 라인별 / 분기 문 별로 파악할 수 있음
- 코드 커버리지로 테스트 품질이 우수한 정도를 파악할 수 없음.
- 테스트가 되지 않고 있는 부분을 파악하기 위해 사용함



코드 커버리지 결과리포트 예시

Line Coverage

- 라인별 코드 수행 여부를 판단
- Line Coverage = **90%** (9/10)
우측 10개 라인 중 9개 코드 수행
- 한계점
Branch 1과 2는 똑같이 중요하지만,
Branch 1 코드수가 길기에
Coverage 수치 상
대부분의 코드가 Cover되는 것으로 오해가능

```
class RoutineChecker {  
    public static void checkEven(int num) {  
        if (num % 2 == 0) {  
            // Branch 1  
            System.out.println("루틴 1");  
            System.out.println("루틴 2");  
            System.out.println("루틴 3");  
            System.out.println("루틴 4");  
            System.out.println("루틴 5");  
            System.out.println("루틴 6");  
            System.out.println("루틴 7");  
            System.out.println("루틴 8");  
        } else {  
            // Branch 2  
            System.out.println("매우 중요한 코드");  
        }  
    }  
  
    public static void main(String[] args) {  
        checkEven(2);  
    }  
}
```

Test Code

Branch Coverage

- 분기문 별 검사
- Branch Coverage = **50%** (1/2)
우측 2개의 분기 중 1개 분기 수행

```
class RoutineChecker {  
    public static void checkEven(int num) {  
        if (num % 2 == 0) {  
            // Branch 1  
            System.out.println("루틴 1");  
            System.out.println("루틴 2");  
            System.out.println("루틴 3");  
            System.out.println("루틴 4");  
            System.out.println("루틴 5");  
            System.out.println("루틴 6");  
            System.out.println("루틴 7");  
            System.out.println("루틴 8");  
        } else {  
            // Branch 2  
            System.out.println("매우 중요한 코드");  
        }  
    }  
  
    public static void main(String[] args) {  
        checkEven(2);  
    }  
}
```

Test Code

리팩토링을 하기 전, Test Case를 얼마나 만들어야 하는가?

Legacy 코드에 대해, 잘 알고 있는 경우

- 중요한 Feature를 모두 검증 할 만큼 필요

Legacy 코드에 대해, 잘 모르는 경우



- 안전장치를 마련하기 위해
Code Coverage 100% 에 가깝도록 테스트 준비를 권장한다.
- Line 보다는 Branch Coverage 100%를 더 권장하지만
실습에는 Line Coverage를 사용한다.

[도전] Unit Test 작성하기

Unit Test를 작성한다.

Code Coverage가 100%가 나오는지 확인한다.

만약 100%가 아니라면,
테스트케이스를 추가하여
100%를 만들어낸다.

Rule

수정해도 영향을 크게 끼치지 않는 부분부터 수정한다.

작은 수정마다 지속적으로 테스트를 한다.

전 세계 모든 개발자들에게 공개된다는 생각으로,
가독성이 좋은 코드로 개선한다.